

Full-Stack Developer Deep-Dive Notes

HTML5 · Advanced CSS · Tailwind CSS · SCSS · JavaScript (Event Loop, Closures, Scope, Web Workers, Async) · TypeScript · React
· Next.js · Testing · Express · Microservices

Prepared for interview & project reference — July 2026

Table of Contents

- 1. HTML & HTML5
- 2. Advanced CSS
- 3. Tailwind CSS
- 4. SCSS (Sass)
- 5. JavaScript Deep Dive: Event Loop, Closures, Scope, Web Workers, Async
- 6. TypeScript
- 7. React
- 8. Next.js
- 9. Testing (Jest, RTL, Cypress)
- 10. Express.js
- 11. Microservices Architecture
- 12. Curated Reference Links

1. HTML & HTML5

1.1 Document structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Page</title>
</head>
<body>...</body>
</html>
```

1.2 Semantic elements (HTML5)

Semantic tags describe meaning, not just layout — important for SEO, accessibility, and maintainability.

Tag	Purpose
<header>, <footer>	Top/bottom sections of page or section
<nav>	Navigation links
<main>	Primary content, one per page
<section>	Thematic grouping of content
<article>	Self-contained content (blog post, card)
<aside>	Tangential content, sidebars
<figure>/<figcaption>	Images/diagrams with captions
<details>/<summary>	Native collapsible widget, no JS needed
<time>	Machine-readable dates via datetime attribute

1.3 Forms & validation

```
<form>
  <input type="email" required placeholder="Email">
  <input type="number" min="1" max="10" step="1">
  <input type="date">
  <input pattern="[0-9]{10}" title="10 digit phone">
</form>
```

New input types: `email`, `url`, `tel`, `date`, `time`, `color`, `range`, `search`. Built-in constraint validation (`required`, `min`, `max`, `pattern`, `minlength`) avoids extra JS for basic checks. Use `:invalid` / `:valid` CSS pseudo-classes to style feedback.

1.4 Key HTML5 APIs

- **Canvas** — pixel-level 2D/3D drawing (`getContext('2d')`), used for charts/games.
- **SVG** — vector graphics, DOM-addressable, scales without quality loss.
- **Web Storage** — `localStorage` (persists), `sessionStorage` (tab lifetime); both ~5-10MB, string-only (JSON.stringify needed).
- **IndexedDB** — client-side NoSQL DB for larger structured data.
- **Drag & Drop API** — `draggable="true"` , `dragstart`/`dragover`/`drop` events.
- **Geolocation API** — `navigator.geolocation.getCurrentPosition()` .
- **<video>/<audio>** — native media playback without plugins.
- **Web Workers & Service Workers** — background threads and offline caching (see Section 5.4).

1.5 Accessibility (a11y) essentials

- Use semantic tags before reaching for ARIA ("no ARIA is better than bad ARIA").
- `alt` text on every meaningful image; empty `alt=""` for decorative ones.
- Label every input: `<label for="id">` or `aria-label` .
- Maintain logical heading order (h1 → h2 → h3, no skipping).
- Keyboard navigability: everything clickable should be focusable and operable via Tab/Enter.

2. Advanced CSS

2.1 Cascade, specificity & inheritance

Specificity order (low → high): element selectors < class/attribute/pseudo-class < ID < inline style < **!important**. Equal specificity → last rule wins (source order).

```
/* specificity example */
div p      /* 0,0,2 */
.card p    /* 0,1,1 */
#main p    /* 1,0,1 -- wins over .card p */
```

2.2 Box model & layout

```
* { box-sizing: border-box; } /* padding/border included in width */
```

Flexbox — one-dimensional layout

```
.container {
  display: flex;
  justify-content: space-between; /* main axis */
  align-items: center;           /* cross axis */
  flex-wrap: wrap;
  gap: 12px;
}
.item { flex: 1 1 200px; } /* grow shrink basis */
```

Grid — two-dimensional layout

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  grid-template-rows: auto 1fr auto;
  gap: 16px;
}
.item { grid-column: span 2; }
```

Flexbox vs Grid: Flex is best for a single row/column of flexible items (navbars, button groups). Grid is best for full page/card layouts needing both rows and columns aligned together.

2.3 Positioning

Value	Behavior
static	Default, normal flow
relative	Offsets from its own normal position; establishes a positioning context for children
absolute	Removed from flow; positioned relative to nearest positioned ancestor
fixed	Positioned relative to viewport, stays on scroll
sticky	Behaves relative until a scroll threshold, then sticks like fixed within its container

2.4 Responsive design

```
@media (max-width: 768px) {
  .grid { grid-template-columns: 1fr; }
}
/* Mobile-first: write base styles for small screens, add min-width overrides */
@media (min-width: 1024px) { .sidebar { display: block; } }
```

Use relative units: **rem** (root-based, scales with user font settings), **%**, **vw/vh**, **clamp(min, preferred, max)** for fluid typography.

```
h1 { font-size: clamp(1.5rem, 4vw, 3rem); }
```

2.5 Custom properties (CSS variables) & theming

```
:root { --primary: #0b5394; --gap: 12px; }
.btn { background: var(--primary); padding: var(--gap); }
[data-theme="dark"] { --primary: #4d9fec; }
```

2.6 Animations & transitions

```
.box {  
  transition: transform 0.3s ease-in-out;  
}  
.box:hover { transform: scale(1.05); }  
  
@keyframes fadeIn {  
  from { opacity: 0; transform: translateY(10px); }  
  to   { opacity: 1; transform: translateY(0); }  
}  
.card { animation: fadeIn 0.5s ease-out; }
```

Prefer animating `transform` and `opacity` — they run on the GPU compositor and don't trigger layout reflow, unlike animating `width/top/left`.

2.7 Pseudo-classes/elements & selectors worth knowing

```
li:nth-child(odd) { background: #f5f5f5; }  
input:focus-visible { outline: 2px solid blue; }  
p::first-line { font-weight: bold; }  
.card:has(img) { border: 1px solid #ccc; } /* parent selector */  
button:not(.disabled) { cursor: pointer; }
```

3. Tailwind CSS

Tailwind is a **utility-first** CSS framework: instead of writing custom class names and CSS rules, you compose small, single-purpose classes directly in markup. This removes context-switching between HTML and CSS files and keeps styling co-located with components — a natural fit for React/JSX.

3.1 Core concepts

```
<button class="bg-blue-600 hover:bg-blue-700 text-white font-semibold
  px-4 py-2 rounded-lg shadow-md transition">
  Submit
</button>
```

3.2 Responsive & state variants

```
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
<input class="border focus:ring-2 focus:ring-blue-400 disabled:opacity-50">
<p class="dark:text-white dark:bg-gray-900">Dark mode aware</p>
```

Breakpoint prefixes (**sm**: **md**: **lg**: **xl**: **2xl**:) are mobile-first — they apply from that width upward.

3.3 Configuration (tailwind.config.js)

```
module.exports = {
  content: ['./src/**/*.js', './src/**/*.jsx', './src/**/*.ts', './src/**/*.tsx'],
  theme: {
    extend: {
      colors: { brand: '#0b5394' },
      fontFamily: { sans: ['Inter', 'sans-serif'] },
    },
  },
  plugins: [require('@tailwindcss/forms')],
};
```

3.4 Reusability

- **@apply** in a CSS file to bundle repeated utility groups into one class for very common patterns.
- Component extraction (React component wrapping a set of classes) is usually preferred over **@apply** for reusability.
- **clsx** or **tailwind-merge** npm packages help conditionally combine classes cleanly in JSX.

4. SCSS (Sass)

SCSS is a CSS preprocessor — superset syntax compiled down to plain CSS, adding programming features CSS itself lacks.

4.1 Variables

```
$primary: #0b5394;
$spacing: 8px;
.btn { padding: $spacing * 2; color: $primary; }
```

4.2 Nesting

```
.card {
  padding: 16px;
  &__title { font-size: 18px; }      // .card__title (BEM pattern)
  &:hover { box-shadow: 0 2px 8px rgba(0,0,0,.1); }
  .icon { margin-right: 4px; }
}
```

4.3 Mixins & functions

```
@mixin flex-center($direction: row) {
  display: flex;
  flex-direction: $direction;
  justify-content: center;
  align-items: center;
}
.hero { @include flex-center(column); }

@function rem($px) { @return ($px / 16) * 1rem; }
h1 { font-size: rem(32); }
```

4.4 Partials & modules

```
// _variables.scss, _mixins.scss
@use 'variables' as v;
.btn { color: v.$primary; }
```

Files prefixed with `_` are partials (not compiled standalone). Modern Sass uses `@use` / `@forward` instead of the older global `@import`.

4.5 Extend & control directives

```
%btn-base { padding: 8px 16px; border-radius: 6px; }
.btn-primary { @extend %btn-base; background: $primary; }

@each $name, $color in (primary: blue, danger: red) {
  .text-#{$name} { color: $color; }
}
```

5. JavaScript Deep Dive

5.1 Scope

Scope determines where a variable is accessible.

- **Global scope** — declared outside any function/block; accessible everywhere.
- **Function scope** — `var` is scoped to the nearest enclosing function, ignoring blocks.
- **Block scope** — `let` / `const` are scoped to the nearest `{ }` (if/for/while/plain block).
- **Lexical scope** — a function's access to variables is determined by where it is *written* in the source code, not where it's called from.

```
function demo() {
  if (true) {
    var a = 1;    // function-scoped -> leaks out of the if-block
    let b = 2;    // block-scoped -> only inside the if-block
  }
  console.log(a); // 1
  console.log(b); // ReferenceError
}
```

Hoisting & the Temporal Dead Zone

```
console.log(x); // undefined (var is hoisted & initialized to undefined)
var x = 5;

console.log(y); // ReferenceError (let/const are hoisted but NOT initialized -> TDZ)
let y = 5;
```

5.2 Closures

A closure is a function that **remembers the variables from its lexical scope**, even after the outer function has finished executing. JS engines keep that outer scope alive in memory as long as an inner function still references it.

```
function makeCounter() {
  let count = 0;           // private state
  return function () {
    count++;               // inner fn "closes over" count
    return count;
  };
}
const counter = makeCounter();
counter(); // 1
counter(); // 2 -- count persisted between calls, but is inaccessible from outside
```

Practical uses of closures

- **Data privacy / module pattern** — emulate private variables before ES classes had `#private` fields.
- **Memoization / caching** — closure holds a cache object across calls.
- **Currying / partial application** — `const add = a => b => a + b;`
- **Event handlers & callbacks** — each handler can "remember" the specific data it was created with (classic React `useState` setters rely on this).

Classic interview trap — closures in loops:

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0); // prints 3,3,3 (var is shared, one binding)
}
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 0); // prints 0,1,2 (let creates a new binding per iteration)
}
```

5.3 The Event Loop

JavaScript is single-threaded — one call stack, one thing running at a time. The event loop is what lets JS handle asynchronous work (timers, network calls, DOM events) without blocking that single thread.

The moving parts

- **Call Stack** — where synchronous function calls execute, LIFO.
- **Heap** — memory for objects/variables.
- **Web APIs / Node APIs** — browser/Node provide timers, DOM events, fetch/XHR, fs, etc., running outside the JS thread.
- **Callback (Task/Macrotask) Queue** — holds callbacks from `setTimeout`, `setInterval`, DOM events, I/O.
- **Microtask Queue** — holds Promise `.then/.catch/.finally` callbacks and `queueMicrotask`.

Execution order rule

After each macrotask (or each top-level script run) finishes, the engine **drains the entire microtask queue** before picking up the next macrotask. This is why Promises "jump ahead of" `setTimeout` even with a 0ms delay.

```
console.log('1: sync');

setTimeout(() => console.log('2: macrotask (setTimeout)'), 0);

Promise.resolve().then(() => console.log('3: microtask (promise)'));

console.log('4: sync');

// Output order: 1, 4, 3, 2
// Sync code always runs first (fills+empties the call stack),
// then ALL microtasks, then the next macrotask.
```

Visual model

```
Call Stack --> runs sync code top to bottom
  |
  v (async fn called)
Web/Node API --> does the work in background (timer, network...)
  |
  v (when done)
Callback Queue (macrotasks) or Microtask Queue (promises)
  |
  v
Event Loop checks: is Call Stack empty?
  -> yes: drain ALL microtasks first, THEN take one macrotask, repeat
```

Node.js adds extra phases (timers, pending callbacks, poll, check via `setImmediate`, close callbacks) plus `process.nextTick`, which runs even before Promise microtasks — worth knowing for backend interviews.

5.4 Web Workers

Web Workers run JavaScript on a **separate background thread**, so CPU-heavy work (image processing, big loops, parsing) doesn't freeze the UI thread. Workers cannot touch the DOM directly and communicate with the main thread only via message passing (structured-clone, not shared memory, unless using `SharedArrayBuffer`).

```
// main.js
const worker = new Worker('worker.js');
worker.postMessage({ n: 45 });
worker.onmessage = (e) => console.log('Result:', e.data);

// worker.js
onmessage = (e) => {
  const result = fib(e.data.n); // expensive work, off the main thread
  postMessage(result);
};
function fib(n) { return n < 2 ? n : fib(n-1) + fib(n-2); }
```

Service Workers are a related but different tool: they sit between the app and the network, enabling offline caching and push notifications (foundation of PWAs) rather than raw computation.

5.5 Asynchronous JavaScript

Callbacks → Promises → async/await

```
// Promise
function fetchUser(id) {
  return new Promise((resolve, reject) => {
    fetch(`/api/users/${id}`)
      .then(res => res.ok ? resolve(res.json()) : reject(new Error('failed')))
      .catch(reject);
  });
}

// async/await -- syntactic sugar over promises, easier to read
async function loadUser(id) {
  try {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) throw new Error('failed');
    const user = await res.json();
    return user;
  } catch (err) {
    console.error(err);
  }
}
```

Combinators

```
Promise.all([p1, p2, p3]);           // fails fast if ANY rejects
Promise.allSettled([p1, p2, p3]);    // waits for all, gives status of each
Promise.race([p1, p2]);              // resolves/rejects with the first to finish
Promise.any([p1, p2]);               // resolves with first fulfillment, ignores rejections unless all fail
```

Key mental model: `await` pauses only the async function it's inside — it does not block the rest of the program, because under the hood the function is suspended and resumed via the microtask queue.

6. TypeScript

6.1 Why TypeScript

TypeScript adds a static type system on top of JavaScript, catching type errors at compile time instead of runtime, improving editor autocomplete, and documenting intent directly in code.

6.2 Basic types & structures

```
let age: number = 25;
let name: string = "Raja";
let tags: string[] = ["js", "react"];
let coords: [number, number] = [10, 20]; // tuple
let status: "idle" | "loading" | "error"; // union / literal type

interface User {
  id: number;
  name: string;
  email?: string; // optional
  readonly createdAt: Date;
}

type Role = "admin" | "user";
type Point = { x: number; y: number };
```

6.3 Functions & generics

```
function add(a: number, b: number): number { return a + b; }

function identity<T>(value: T): T { return value; }
const num = identity<number>(5);

interface ApiResponse<T> {
  data: T;
  error: string | null;
}

async function getUsers(): Promise<ApiResponse<User[]>> { /* ... */ }
```

6.4 Utility types (frequently used)

```
Partial<User>      // all properties optional
Required<User>    // all properties required
Pick<User, "id"|"name"> // subset of keys
Omit<User, "email"> // exclude keys
Readonly<User>    // all properties readonly
Record<string, number> // dictionary type
```

6.5 Classes & interfaces together

```
interface Shape { area(): number; }

class Circle implements Shape {
  constructor(private radius: number) {}
  area(): number { return Math.PI * this.radius ** 2; }
}
```

6.6 Type narrowing

```
function printId(id: string | number) {
  if (typeof id === "string") {
    console.log(id.toUpperCase()); // TS knows it's a string here
  } else {
    console.log(id.toFixed(2));
  }
}
```

7. React

7.1 Core mental model

React renders UI as a function of state: `UI = f(state)`. Components re-render when their state or props change; React diffs the returned virtual DOM tree and applies the minimal set of real DOM updates (reconciliation).

7.2 Hooks

```
const [count, setCount] = useState(0);

useEffect(() => {
  const id = setInterval(() => setCount(c => c + 1), 1000);
  return () => clearInterval(id); // cleanup on unmount/dep change
}, []); // deps array -> run once on mount

const value = useMemo(() => expensiveCalc(count), [count]); // memoize a computed value
const stableFn = useCallback(() => doSomething(count), [count]); // memoize a function reference

const ThemeContext = createContext();
const theme = useContext(ThemeContext);

const [state, dispatch] = useReducer(reducer, initialState); // Redux-like local state
const inputRef = useRef(null); // mutable value that survives renders, doesn't trigger re-render
```

Rules of Hooks

- Only call hooks at the top level — never inside loops, conditions, or nested functions.
- Only call hooks from React function components or custom hooks.
- Custom hooks (functions starting with `use`) let you extract and share stateful logic between components.

7.3 Performance patterns

```
const MemoChild = React.memo(function Child({ value }) {
  return <div>{value}</div>; // skips re-render if props are shallow-equal
});
```

- Avoid creating new object/array/function literals inline as props unless memoized — they break `React.memo` shallow comparison.
- Lift state only as high as necessary; colocate state near where it's used to limit re-render scope.
- Use keys correctly in lists — stable, unique IDs, never array index if list order can change.

7.4 State management (RTK Query, since it's in your stack)

```
export const apiSlice = createApi({
  reducerPath: 'api',
  baseQuery: fetchBaseQuery({ baseUrl: '/api' }),
  endpoints: (builder) => ({
    getUsers: builder.query({ query: () => '/users' }),
    addUser: builder.mutation({
      query: (body) => ({ url: '/users', method: 'POST', body }),
      invalidatesTags: ['User'],
    }),
  }),
});
export const { useGetUsersQuery, useAddUserMutation } = apiSlice;
```

RTK Query auto-generates hooks that handle caching, re-fetching, loading/error states — removing most hand-written `useEffect` data-fetching boilerplate.

8. Next.js

8.1 Rendering strategies

Strategy	When it runs	Use case
SSR (Server-Side Rendering)	On every request, on the server	Frequently changing, personalized data
SSG (Static Site Generation)	At build time	Marketing pages, blogs — rarely changes
ISR (Incremental Static Regeneration)	Static, but regenerates after a revalidate interval	Product pages, semi-frequent updates
CSR (Client-Side Rendering)	In the browser after JS loads	Highly interactive, user-specific dashboards

8.2 App Router basics (Next.js 13+)

```
app/  
  layout.tsx      // shared shell (nav, providers) for all pages below  
  page.tsx        // route: "/"  
  about/page.tsx  // route: "/about"  
  users/[id]/page.tsx // dynamic route: "/users/5"  
  api/users/route.ts // API endpoint: "/api/users"  
  
// Server Component (default) -- can fetch data directly, runs only on server  
export default async function Page() {  
  const res = await fetch('https://api.example.com/data', { cache: 'no-store' });  
  const data = await res.json();  
  return <div>{data.title}</div>;  
}  
  
// Client Component -- needed for hooks/interactivity  
'use client';  
export default function Counter() {  
  const [n, setN] = useState(0);  
  return <button onClick={() => setN(n+1)}>{n}</button>;  
}
```

8.3 Data fetching & caching

```
fetch(url, { cache: 'force-cache' }); // default, like SSG  
fetch(url, { cache: 'no-store' });    // like SSR, always fresh  
fetch(url, { next: { revalidate: 60 } }); // ISR, refresh every 60s
```

8.4 API routes / Route Handlers

```
// app/api/users/route.ts  
export async function GET() {  
  const users = await db.user.findMany();  
  return Response.json(users);  
}  
  
export async function POST(request: Request) {  
  const body = await request.json();  
  const user = await db.user.create({ data: body });  
  return Response.json(user, { status: 201 });  
}
```

9. Testing

9.1 Testing pyramid

Unit tests (many, fast, isolated) → Integration tests (fewer, test components/modules together) → End-to-End tests (few, slow, test full user flows in a real browser).

9.2 Jest — unit testing

```
function sum(a, b) { return a + b; }

test('adds two numbers', () => {
  expect(sum(2, 3)).toBe(5);
});

describe('user validation', () => {
  it('rejects invalid email', () => {
    expect(() => validate({email:'bad'})).toThrow();
  });
});

// Mocking
jest.mock('./api');
api.getUser.mockResolvedValue({ id: 1, name: 'Raja' });
```

9.3 React Testing Library — component testing

```
import { render, screen, fireEvent } from '@testing-library/react';

test('increments counter on click', () => {
  render(<Counter />);
  const button = screen.getByRole('button', { name: /increment/i });
  fireEvent.click(button);
  expect(screen.getByText('1')).toBeInTheDocument();
});
```

RTL philosophy: test components the way a user interacts with them (by visible text/roles), not by internal implementation details like state variables.

9.4 End-to-end (Cypress / Playwright)

```
// Cypress
cy.visit('/login');
cy.get('input[name=email]').type('raja@test.com');
cy.get('input[name=password]').type('secret123');
cy.get('button[type=submit]').click();
cy.url().should('include', '/dashboard');
```

9.5 Backend/API testing (supertest with Express)

```
const request = require('supertest');
const app = require('../app');

test('GET /api/users returns 200', async () => {
  const res = await request(app).get('/api/users');
  expect(res.statusCode).toBe(200);
  expect(Array.isArray(res.body)).toBe(true);
});
```

10. Express.js

10.1 Basic server & routing

```
const express = require('express');
const app = express();
app.use(express.json());

app.get('/api/users/:id', async (req, res) => {
  const user = await User.findById(req.params.id);
  if (!user) return res.status(404).json({ message: 'Not found' });
  res.json(user);
});

app.listen(5000, () => console.log('Server running on 5000'));
```

10.2 Middleware

Middleware functions run in order, each receiving `(req, res, next)`, and can modify the request, end the response, or call `next()` to pass control along.

```
function authMiddleware(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'No token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ message: 'Invalid token' });
  }
}

app.get('/api/profile', authMiddleware, getProfile);
```

10.3 Centralized error handling

```
app.use((err, req, res, next) => { // 4 args = error-handling middleware
  console.error(err.stack);
  res.status(err.statusCode || 500).json({ message: err.message || 'Server error' });
});

// trigger it from anywhere with:
app.get('/x', (req, res, next) => {
  try { /* ... */ } catch (err) { next(err); } // forwards to error handler above
});
```

10.4 Structuring a real app

```
src/
  routes/userRoutes.js
  controllers/userController.js
  models/User.js
  middleware/auth.js
  config/db.js
  server.js
```

11. Microservices Architecture

11.1 Monolith vs Microservices

	Monolith	Microservices
Deployment	One unit	Independent services, deployed separately
Scaling	Scale entire app	Scale only the service under load
Tech stack	Usually one stack	Each service can use its own stack/DB
Complexity	Simple to start	Complex: networking, data consistency, observability
Failure isolation	One bug can crash whole app	A failing service can be isolated (with circuit breakers)

11.2 Core building blocks

- **API Gateway** — single entry point that routes requests to the right service, handles auth, rate limiting (e.g. Kong, AWS API Gateway, or a simple Express gateway).
- **Service discovery** — how services find each other's network location (Consul, Eureka, or Kubernetes DNS).
- **Inter-service communication** — synchronous (REST/gRPC over HTTP) or asynchronous (message queues like RabbitMQ/Kafka for event-driven flows).
- **Database per service** — each service owns its own data store; no other service touches it directly, only via that service's API.
- **Circuit breaker** — stops calling a failing downstream service temporarily, preventing cascading failures (e.g. resilience4j, opossum for Node).

11.3 Communication patterns

```
// Synchronous REST call between services (order-service calling user-service)
const res = await axios.get(`http://user-service:5001/api/users/${userId}`);

// Asynchronous event with a message broker (order-service publishes, others react)
channel.publish('orders', 'order.created', Buffer.from(JSON.stringify(order)));
// notification-service and inventory-service subscribe independently
```

Event-driven (async) communication decouples services in time — the publisher doesn't need the subscriber to be online — at the cost of eventual consistency instead of immediate consistency.

11.4 Containers & orchestration

Each microservice is typically packaged as a Docker container, with Kubernetes (or Docker Compose for local dev) handling deployment, scaling, health checks, and networking between containers.

```
# docker-compose.yml (simplified)
services:
  user-service:
    build: ./user-service
    ports: ["5001:5001"]
  order-service:
    build: ./order-service
    ports: ["5002:5002"]
    depends_on: [user-service]
```


12. Curated Reference Links

HTML / HTML5

- MDN HTML: <https://developer.mozilla.org/en-US/docs/Web/HTML>
- HTML5 semantic elements: <https://developer.mozilla.org/en-US/docs/Web/HTML/Element>
- web.dev Learn HTML: <https://web.dev/learn/html>

CSS

- MDN CSS: <https://developer.mozilla.org/en-US/docs/Web/CSS>
- CSS Flexbox guide: <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
- CSS Grid guide: <https://css-tricks.com/snippets/css/complete-guide-grid/>
- web.dev Learn CSS: <https://web.dev/learn/css>

Tailwind CSS

- Official docs: <https://tailwindcss.com/docs>
- Tailwind Play (online sandbox): <https://play.tailwindcss.com>

SCSS / Sass

- Official Sass docs: <https://sass-lang.com/documentation/>
- Sass guide: <https://sass-lang.com/guide/>

JavaScript

- MDN JavaScript: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- Event loop visualized (Philip Roberts talk): <https://www.youtube.com/watch?v=8aGhZQkoFbQ>
- JavaScript.info (closures, scope, async): <https://javascript.info/>
- MDN Web Workers: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API
- MDN Promises & async/await: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises

TypeScript

- Official handbook: <https://www.typescriptlang.org/docs/handbook/intro.html>
- TypeScript utility types: <https://www.typescriptlang.org/docs/handbook/utility-types.html>

React

- Official docs (new react.dev): <https://react.dev/learn>
- Hooks reference: <https://react.dev/reference/react>
- RTK Query docs: <https://redux-toolkit.js.org/rtk-query/overview>

Next.js

- Official docs: <https://nextjs.org/docs>
- App Router docs: <https://nextjs.org/docs/app>
- Data fetching & caching: <https://nextjs.org/docs/app/building-your-application/data-fetching>

Testing

- Jest docs: <https://jestjs.io/docs/getting-started>
- React Testing Library docs: <https://testing-library.com/docs/react-testing-library/intro/>
- Cypress docs: <https://docs.cypress.io/>
- Playwright docs: <https://playwright.dev/docs/intro>

Express

- Official docs: <https://expressjs.com/>
- Express routing guide: <https://expressjs.com/en/guide/routing.html>
- Error handling guide: <https://expressjs.com/en/guide/error-handling.html>

Microservices

- Microservices.io patterns: <https://microservices.io/patterns/index.html>
- Docker docs: <https://docs.docker.com/>
- Kubernetes docs: <https://kubernetes.io/docs/home/>